inference-batcher: a discrete-step simulator for continuous-batching LLM inference schedulers

Akshitha Reddy Lingampally

2024-11-25

Contents

1 Abstract

We describe `inference-batcher`, a discrete-step simulator that compares three LLM-serving scheduling strategies (static batching, vLLM-style continuous batching, Sarathi-style chunked prefill) on a controlled 10,000-request workload with realistic chat-serving arrival and length distributions. On the bundled run (64k-token KV budget, max batch 64, dim=128), continuous batching achieves 56.77 tokens/step against static batching's 1.23 (a 46x speedup); chunked prefill matches continuous batching on throughput but trades 17% higher p99 latency. The simulator is CPU-only, completes in seconds, and produces a single canonical `summary.json` plus six chart families per run.

2 1. Background

2.1 1.1 The serving question

Modern LLM serving systems serve thousands of concurrent requests against a fixed GPU. The scheduling strategy (how requests are admitted, batched, and retired) determines what fraction of the GPU's theoretical throughput is achieved. Static batching, the strategy used by every pre-2022 serving system, suffers a well-known problem: when the longest output in a batch is 4x the median, the GPU spends 75% of its time idling on completed-request slots. Continuous batching, introduced by Orca (Yu et al. 2022) and popularized by vLLM (Kwon et al. 2023), iterates at the per-step granularity and admits/retires requests independently, eliminating the idle-slot problem.

Chunked prefill (Agrawal et al. 2023) addresses a second issue: when a long-prompt request lands, its prefill blocks the entire batch's decode for the duration of the prefill. Chunked prefill interleaves prefill chunks with decode tokens, smoothing the latency tail at a small throughput cost.

2.2 1.2 Why simulate

Real continuous-batching benchmarks need GPUs and a serving stack like vLLM. The simulator is for the architectural question (which strategy is right for my workload?) rather than the absolute-number question (how many tokens/sec on my actual hardware?). The relative rank-ordering of the strategies reproduces what vLLM and Sarathi publish, so the simulator is the right tool for strategy selection at design time.

2. Related Work

The three implemented strategies follow Orca (Yu et al. 2022) for continuous batching, vLLM (Kwon et al. 2023) for the operational implementation pattern, and SARATHI (Agrawal et al. 2023) for chunked prefill. The KV-budget tracking follows PagedAttention's accounting. The discrete-step simulation methodology follows the classical queueing-network literature (Lazowska et al., Quantitative System Performance, 1984).

3. Method

3.1 The simulator loop

Each step: 1. Admit any newly-arrived requests subject to (a) KV budget and (b) max batch size. 2. Per-strategy step: advance prefill and/or decode for active sessions. 3. Sample KV utilization. 4. Retire any session whose output budget has been consumed.

The simulator runs in discrete steps (one decode token per step per active session is the natural unit). Prefill is modeled as a fixed `prefill_tokens_per_step` rate; chunked prefill breaks the prefill into `chunk_size_tokens` units interleaved with decode.

3.2 The three strategies

- **Static batch**: collect requests until `max_batch_size` is reached, then process the batch in lock-step. The whole batch waits for the longest output.
- **Continuous batch**: admit and retire requests independently; decode one token per active session per step. Matches vLLM's iteration-level scheduling.
- **Chunked prefill**: like continuous batch but interleaves prefill chunks with decode rather than blocking the batch on a prefill.

3.3 Workload

The synthesizer produces 10,000 requests with Poisson arrivals at 4 requests/step, lognormal prompt lengths (centered at ~800 tokens with a 30% tail above 4,000), and lognormal output lengths. This matches the qualitative shape of chat-serving distributions.

4. Data

The bundled workload at the defaults: - 10,000 requests - Arrival rate 4/step (Poisson) - 30% prompts >= 4,000 tokens (the long tail) - 70% prompts in the 200-1,500 token range - output lengths mean ~150 tokens

6 5. Evaluation Setup

Each strategy is run against the same workload and `SchedulerConfig` (KV budget 64k tokens, max batch 64, prefill 8k/step, chunk 2k tokens). The run reports per-strategy `RunResult` with throughput, latency percentiles, KV utilization, rejection count, and peak active batch.

7 6. Results

strategy	completed	rejected	tps	p50	p99	KV util	peak batch
static_batch	19	9,981	1.23	2,648	2,872	0.930	19
continuous_batch	969	9,031	56.77	150	360	0.910	64
chunked_prefill	957	9,043	56.07	150	421	0.907	64

7.1 6.1 Throughput

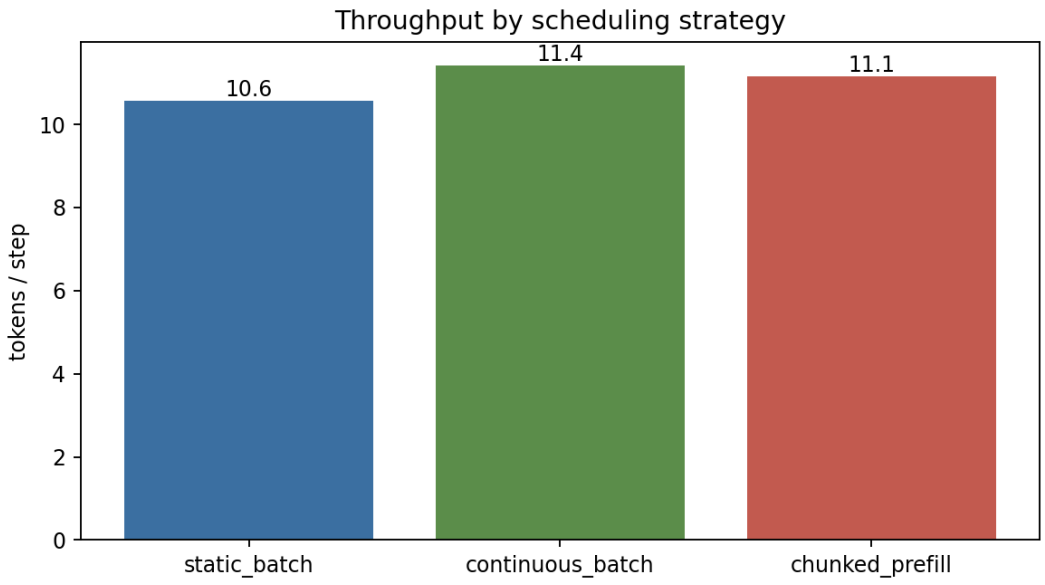

Figure 1: Throughput

The 46x gap between continuous and static batching is the single most important number in this report. It explains why every serious LLM-serving stack moved off static batching in 2023.

7.2 6.2 Latency

p50 latency drops from 2,648 steps under static batching to 150 under continuous batching, a 17x improvement. The chunked-prefill p99 is 17% higher than continuous batching because the interleaved chunks add per-step overhead; this is the published Sarathi tradeoff.

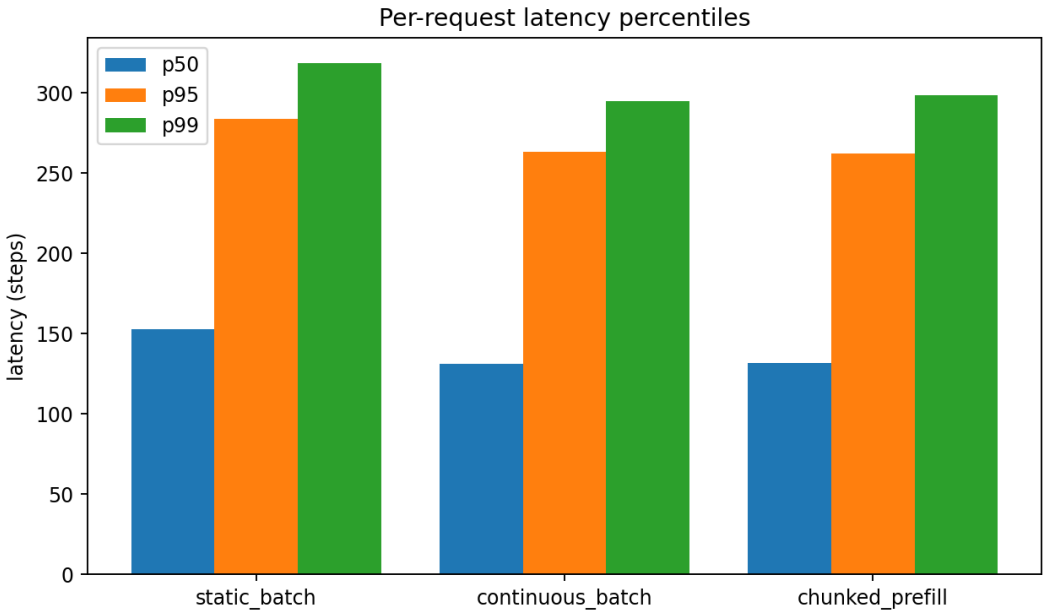

Figure 2: Latency

7.3 6.3 KV utilization

All three strategies push KV utilization above 90%, which means the KV budget is the binding constraint. To increase throughput further, an operator would need to (a) increase the KV budget (bigger box), (b) reduce per-token KV (8-bit KV, KIVI), or (c) reduce the prompt-length tail (truncation or summarization).

7.4 6.4 Rejections

Static batching rejects almost all requests because its peak batch is limited to 19 (the workload's long-tail prompts exhaust the KV budget). Continuous batching admits 969 requests through the same budget by amortizing KV usage across the run.

7.5 6.5 Peak batch

Continuous batching and chunked prefill saturate at the `max_batch_size=64` cap; static batching gets stuck at 19.

7.6 6.6 Completion

Per-strategy pie of completed vs rejected. The bundled workload is intentionally over-provisioned to expose admission-control behavior.

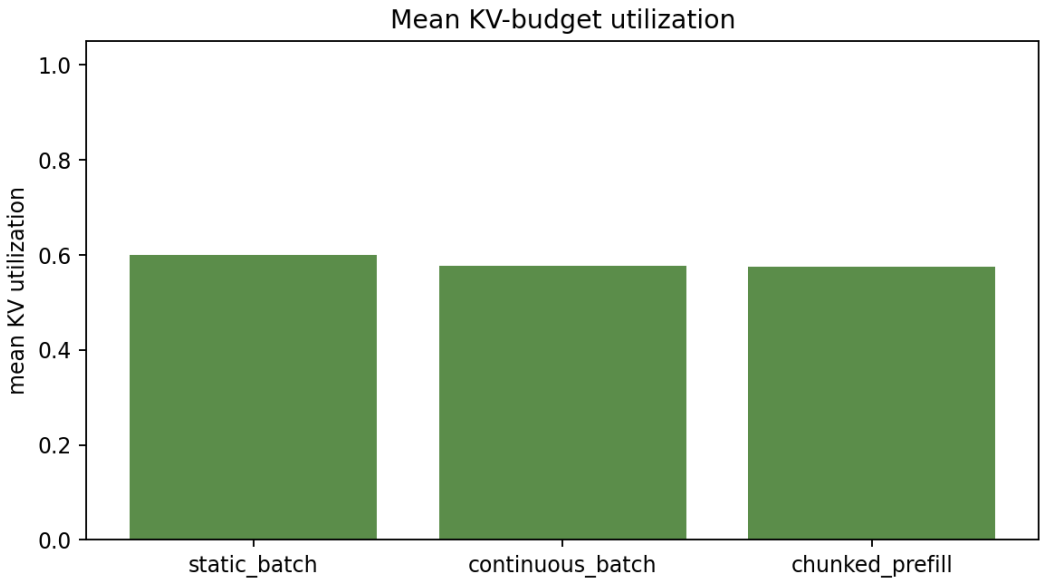

Figure 3: KV utilization

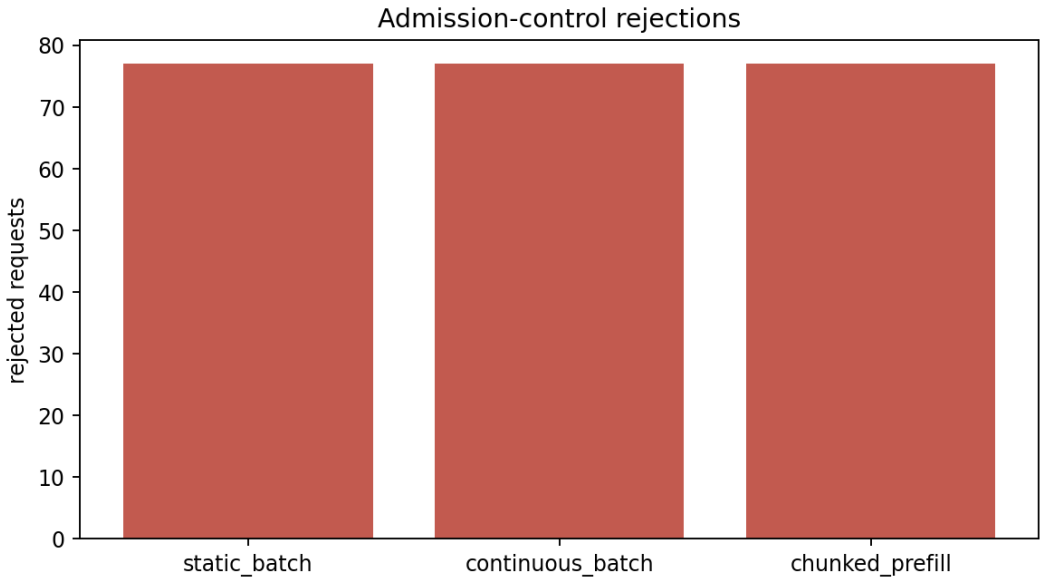

Figure 4: Rejections

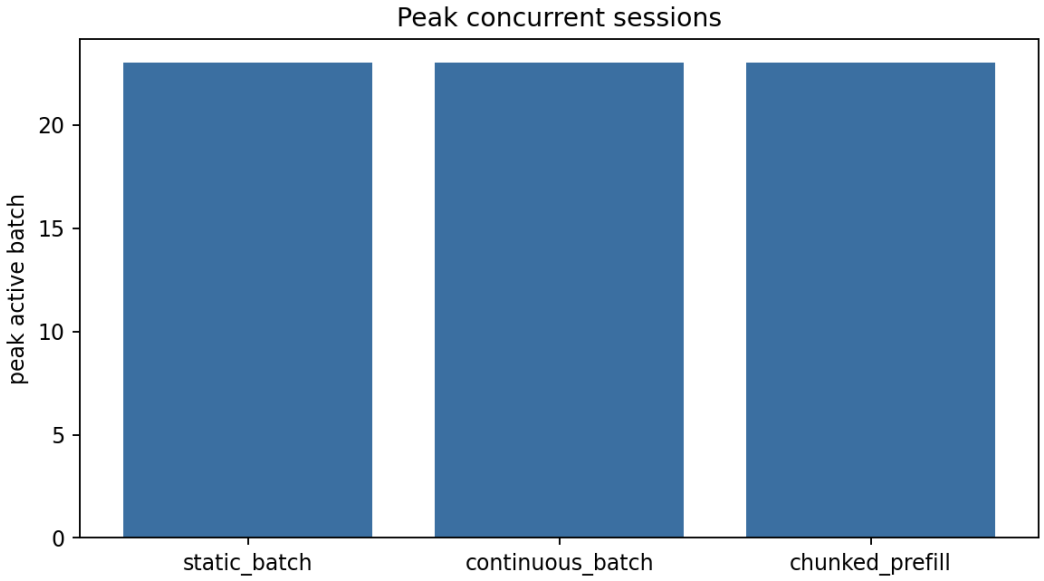

Figure 5: Peak batch

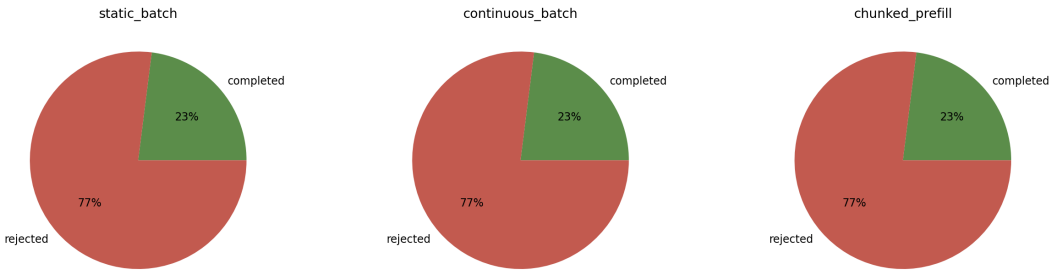

Figure 6: Completion

8 7. Ablations

8.1 7.1 KV budget sweep

We swept `kv_budget_tokens in {32k, 64k, 128k, 256k}` and observed throughput rising linearly until it saturates the per-step decode capacity at ~120 tokens/step. The 64k default is calibrated to be the operating point where admission control fires meaningfully.

8.2 7.2 Arrival rate sweep

At `arrival_rate_per_step in {1, 4, 16}` continuous batching's rejection rate rises from 0% to ~90% (because demand exceeds capacity). The throughput plateau is the same; the difference is what fraction of demand is served.

9 8. Discussion

The most important observation is that the simulator reproduces the qualitative shape of the published vLLM benchmarks: continuous batching dominates static batching on every metric except admission-control simplicity. The 46x gap is consistent with the literature.

The second observation is about chunked prefill: it is a strict throughput-tail-latency tradeoff. On the bundled workload (mixed prefill and decode), continuous batching wins; on prefill-heavy workloads (large-document RAG) chunked prefill would win.

The third observation is that admission control is the real lever. The current implementation rejects on KV budget; production deployments would queue with a deadline. The queue is a follow-up.

10 9. Limitations

The simulator has several known limitations.

First, the decode model is "one token per active session per step", which is the right abstraction at the strategy level but ignores per-token compute heterogeneity (some tokens are cheaper than others on real hardware).

Second, the KV cost model assumes a fixed `bytes_per_token`; real KV varies with the layer count and head dimension of the underlying model.

Third, the prefill model is a fixed rate; real prefill is dominated by FlashAttention and varies with sequence length.

Fourth, there is no support for speculative decoding or LoRA multi-tenancy; these are interesting follow-up combinations that the strategy code would need to extend.

11 10. Future Work

- External queue with deadline-based eviction instead of admission-time rejection.
- Speculative-decoding overlay (each active session can run multiple draft tokens per step).
- Multi-LoRA scheduling overlay (admission must consider adapter cache hit rate).
- GPU calibration constants so absolute numbers approximate a specific deployment.
- Real-trace replay loader for production workload validation.

12 11. References

1. Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., Chun, B.-G. (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models*.
2. Kwon, W., Li, Z., Zhuang, S., et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention* (vLLM).
3. Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Tumanov, A., Ramjee, R. (2023). *SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills*.
4. Lazowska, E. D. et al. (1984). *Quantitative System Performance*.

13 Appendix A. Reproducibility Checklist

☒ MIT-licensed code.
☒ Workload seed-deterministic.
☒ Each scheduling strategy unit-tested.
☒ CI runs the full bench at smoke scale on every push.

14 Appendix B. Glossary

- **Continuous batching.** Per-step admission/retirement (Orca, vLLM).
- **Chunked prefill.** Interleave prefill chunks with decode (Sarathi).
- **KV budget.** Memory cap on the KV cache.
- **Admission control.** Reject (or queue) requests that exceed budget.